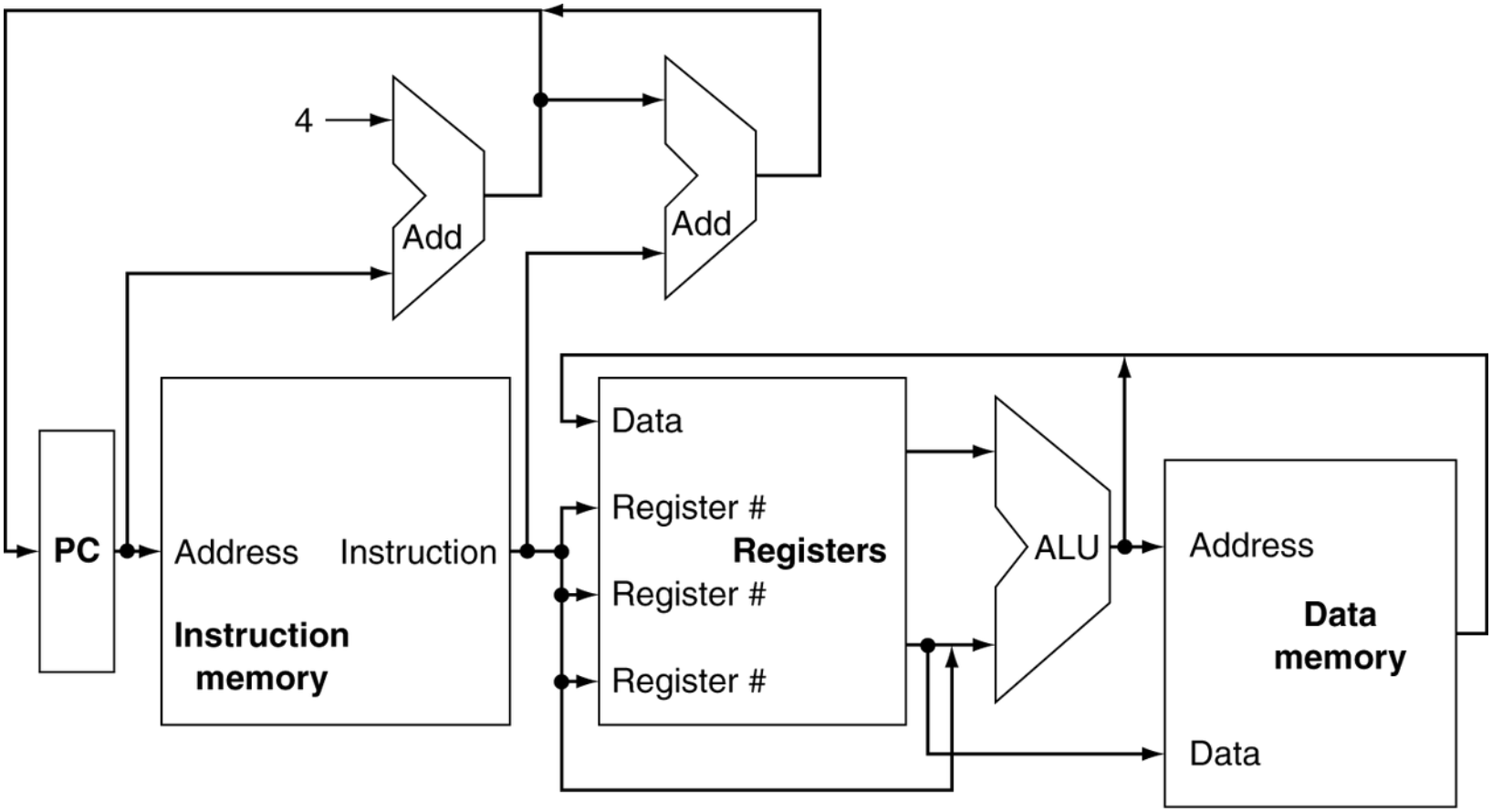
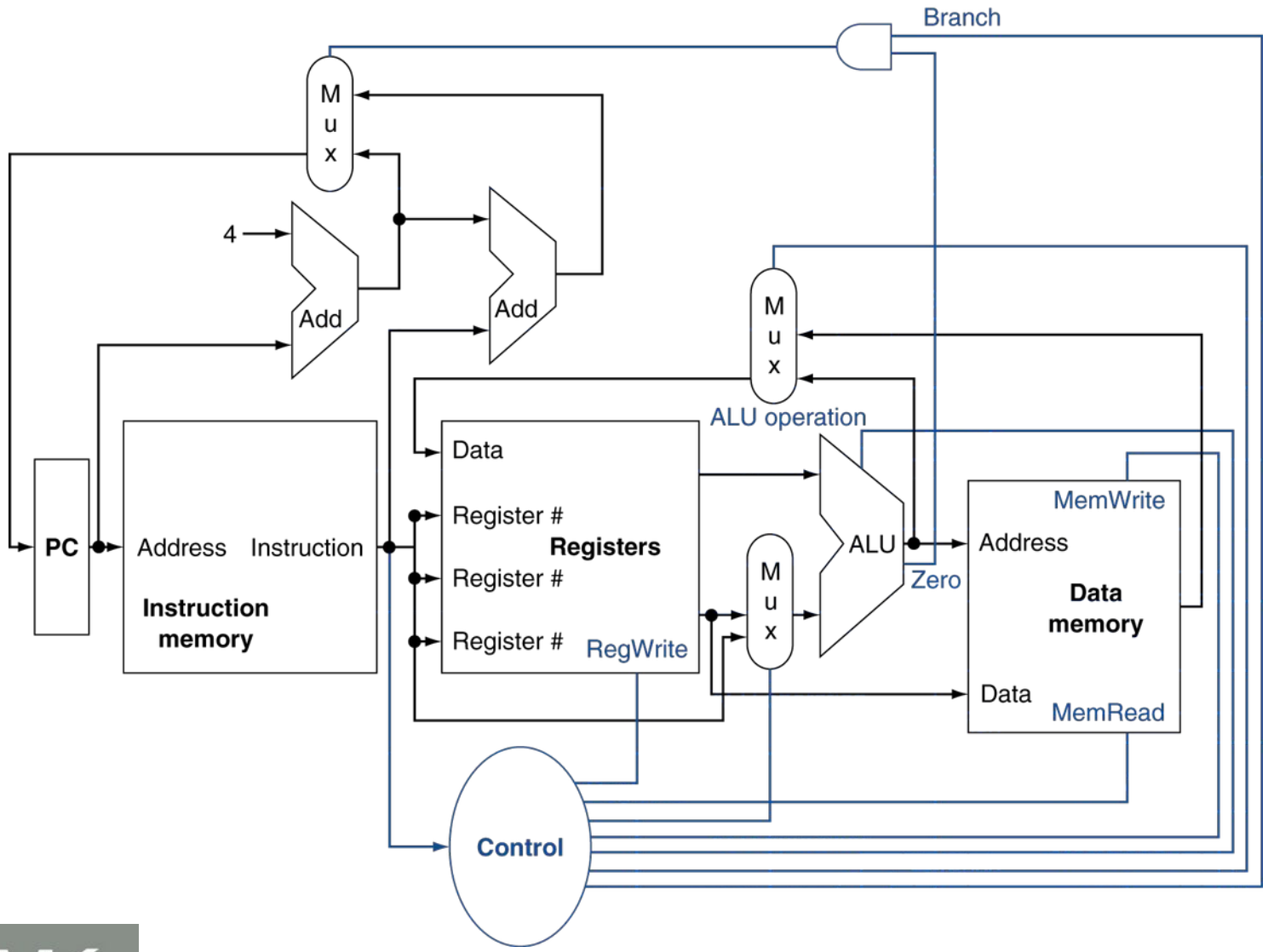
Chapter 3

The Processor

CPU Overview (Recap)



Control (Recap)

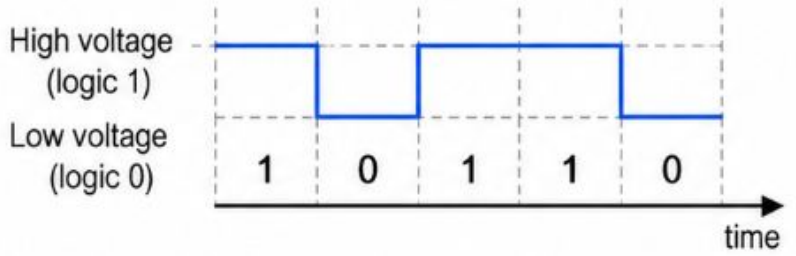


Logic Design Basics

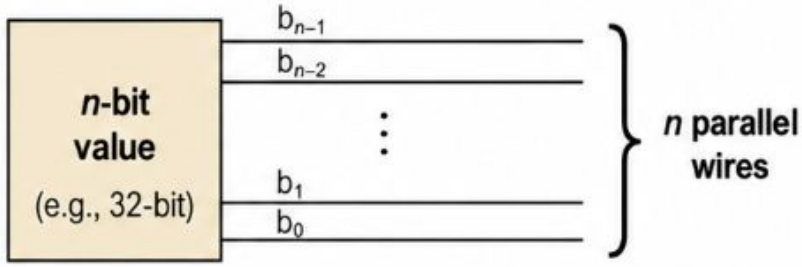
- Information encoded in binary
  - Low voltage = 0, High voltage = 1
  - One wire per bit
  - Multi-bit data encoded on multi-wire buses
- Combinational element
  - Operate on data
  - Output is a function of input
- State (sequential) elements
  - Store information

Binary in Hardware

- **Each bit is represented by an electrical signal.**
  - Low voltage represents logic 0.
  - High voltage represents logic 1.



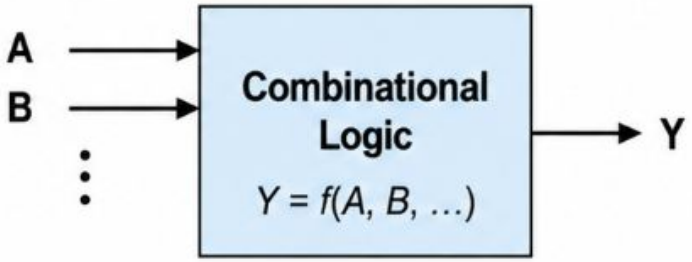
- **A multi-bit value is represented by a set of wires.**
  - An *n*-bit value uses *n* parallel wires.
  - Each wire carries one bit (0 or 1).



Item	Size	Hardware Representation
Instruction	32 bits	32 parallel wires carrying the instruction bits
Program Counter (PC)	32 bits	32 parallel wires carrying the PC value
Register data	32 bits	32 parallel wires carrying the register value

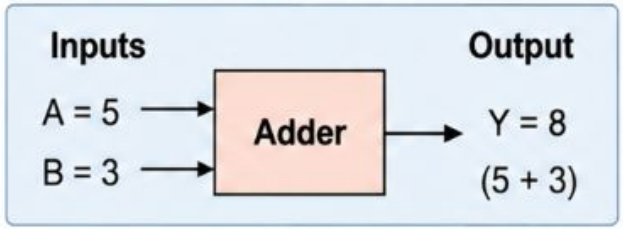
Combinational Logic

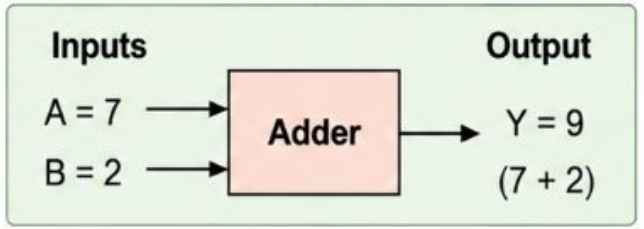
- **Combinational logic processes input signals to produce an output signal.**
  - Receives one or more input signals.
  - Performs a logic or arithmetic operation.
  - The output depends only on the **current input values**.



- **The same inputs always produce the same output.**
  - The output does not depend on past inputs or any stored information.

Example: Adder



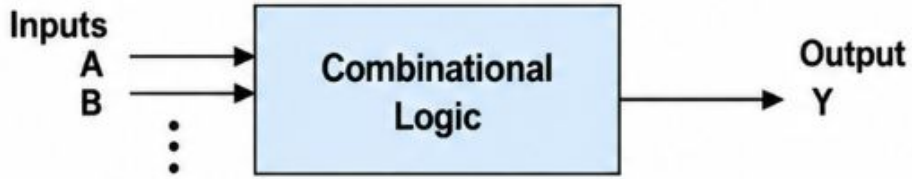


- **Examples inside a processor**

Adder	**Multiplexer**	**ALU**
(e.g., for arithmetic operations)	(e.g., to select one of multiple inputs)	(performs arithmetic and logic operations)

State Elements

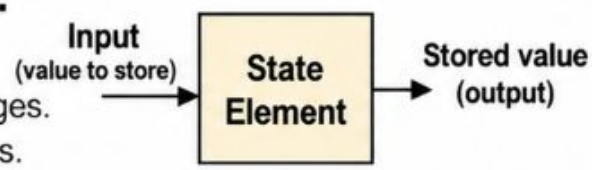
- **Combinational logic can process data, but it does not remember information.**
  - The output depends only on the **current input values**.
  - When the inputs change, a new output is produced immediately.



No stored information
Output depends only on the current input values.

- **A processor must also remember information.**
  - The program counter (PC) must remember the address of the next instruction.
  - Registers and memory must retain data values for use in later instructions.

- **State elements store information.**
  - Store a value.
  - Retain the value even when the input changes.
  - Provide the stored value for later operations.



Stores information
The output can retain a value over time, even if the input changes.

Processor Element	What It Stores	Note
Program Counter	Instruction address	Keeps track of which instruction to fetch
Registers	Data values	Store operands and intermediate results
Memory	Instructions and data	Stores a large amount of processor-acce ssible information

State Elements

- **Combinational logic can process data, but it does not remember information.**
  - The output depends only on the **current input values**.
  - When the inputs change, a new output is produced immediately.



No stored information
Output depends only on the current input values.

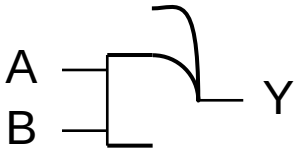
Q1. A processor needs to perform the following three tasks:
A. Add two 32-bit values.
B. Keep the result for use by the next instruction.
C. Transfer the 32-bit result between hardware elements.
For each task, identify what is needed: a combinational element, a state element, or wires carrying binary signals.

A1.
Add two values → Combinational element
Keep the result → State element
Transfer the 32-bit result → Wires carrying binary signals

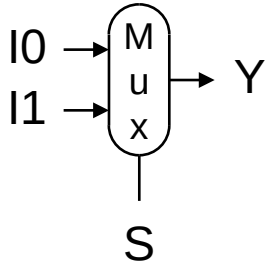
Memory	Instructions and data	ssible information

Combinational Elements

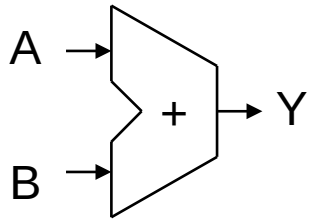
- AND-gate
  - Y = A & B



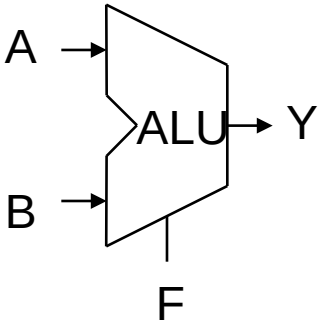
- Multiplexer
  - Y = S ? I1 : I0



- Adder
  - Y = A + B



- Arithmetic/Logic Unit
  - Y = F(A, B)



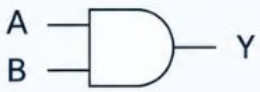
Combinational Elements

Element	Function	Example / Expression	Use in a CPU
Logic Gate	Performs basic logic operations	Y = A AND B	Bitwise AND, OR, XOR operations
Adder	Adds binary values	Y = A + B	PC increment, address calculation, arithmetic
Comparator	Compares two binary values	A = B, A < B, A > B	Value comparison, branch decisions
Multiplexer	Selects one of several inputs	Y = S ? I_1 : I_0	Selecting data or input paths
Decoder	Activates an output based on a binary input	n-bit input → 2^n outputs	Instruction/control decoding, hardware selection

Logic Foundations in Disguise

Logic

Performs basic logic operations (AND, OR, NOT, XOR, etc.).



Example / Expression

Y = A AND B

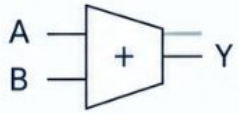
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Use in a CPU

- Bitwise AND, OR, XOR operations

Arithmetic

Performs arithmetic operations (addition, subtraction, comparison, etc.).



Example / Expression

Y = A + B

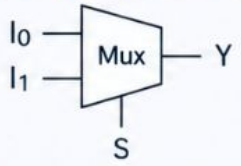
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Use in a CPU

- PC increment
- Address calculation
- Arithmetic operations

Selection

Selects one of multiple inputs based on a control signal.



Example / Expression

Y = S ? I1 : I0

S	I_0	I_1	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

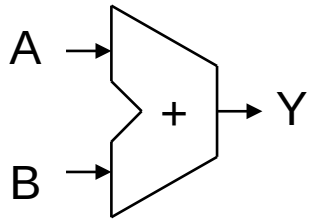
Use in a CPU

- Selecting data or input paths

Different Functions (Logic / Arithmetic / Selection) → **Same Foundation (Boolean Logic)** → **Implemented with Binary Logic**

Combinational Elements

- AND-gate
  - Y = A & B
- Adder
  - Y = A + B



Q2. A CPU needs to perform the following operations:

- **Add two values**
- **Select one of two input values**
- **Compare whether two values are equal**

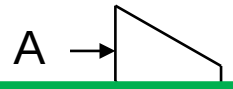
Which combinational element is suitable for each operation?

A2.

- **Adder → performs addition**
- **Multiplexer (Mux) → selects one of multiple inputs**
- **Comparator → compares two values**

Combinational Elements

- AND-gate
- Adder



Q3. A 1-bit Half Adder adds two binary inputs, A and B. Complete its truth table and express the outputs Sum and Carry using AND, OR, NOT, and XOR:

1-bit Half Adder

Inputs: A, B **Outputs: Sum, Carry**

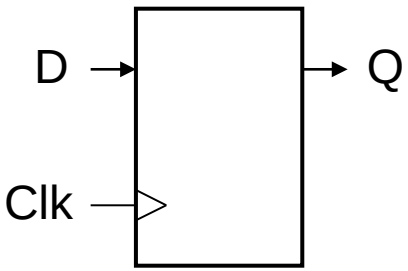
A3.

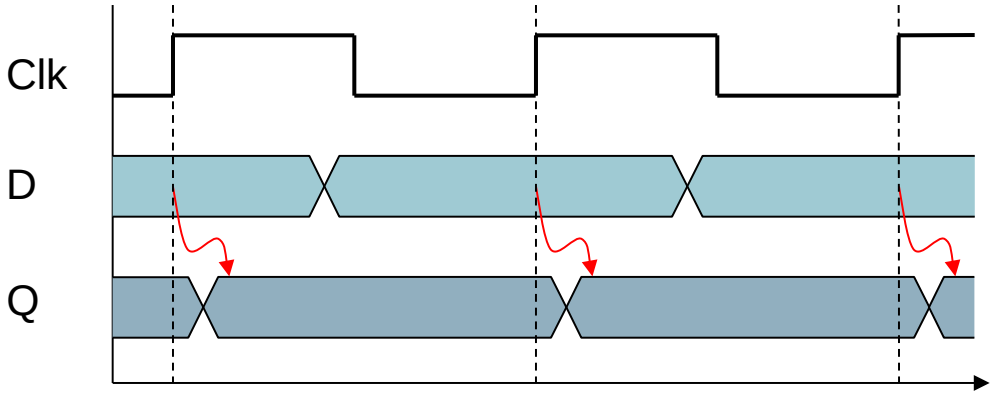
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Sum = A XOR B, Carry = A AND B

Sequential Elements

- Register: stores data in a circuit
  - Uses a clock signal to determine when to update the stored value
  - Edge-triggered: update when Clk changes from 0 to 1





Sequential Elements

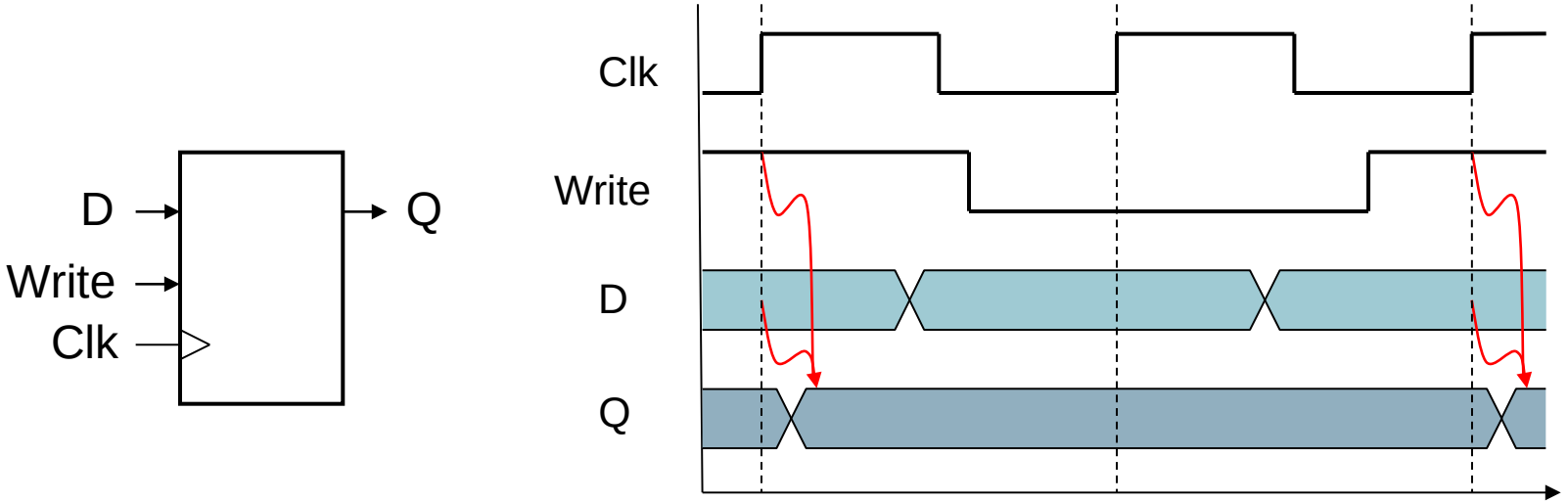
- Register: stores data in a circuit

	Combinational	Sequential
Output depends on	Current inputs	Current inputs + stored state
Stores information?	No	Yes
Typical examples	Adder, Mux	Register, PC
Time matters?	Usually no state/history	Yes

Sequential Element	Main Function	CPU Example
Register	Stores a multi-bit value	General-purpose registers
Program Counter (PC)	Stores an instruction address	Address of current/next instruction
Register File	Stores multiple registers	CPU operand storage
Memory	Stores a large amount of information	Instructions and data

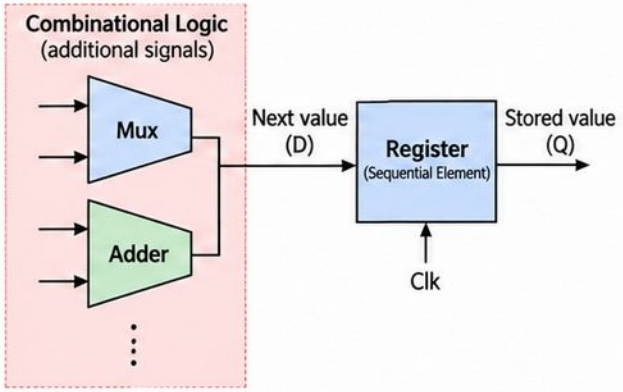
Sequential Elements

- Register with write control
  - Only updates on clock edge when write control input is 1
  - Used when stored value is required later



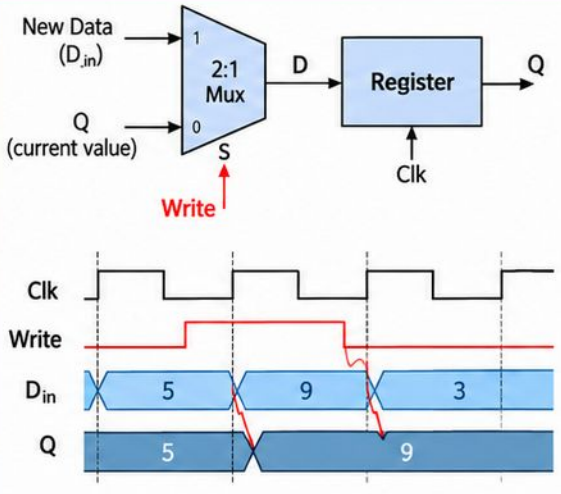
Sequential Logic Design

Sequential Elements with Additional Logic (1/5)



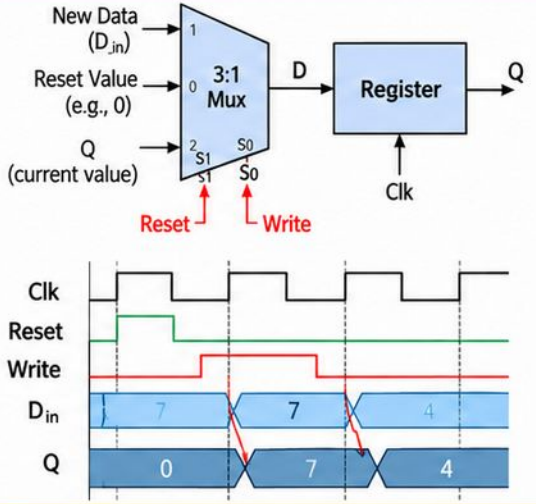
Sequential elements often work with combinational logic to determine what value to store.

Register with Write Control (2/5)



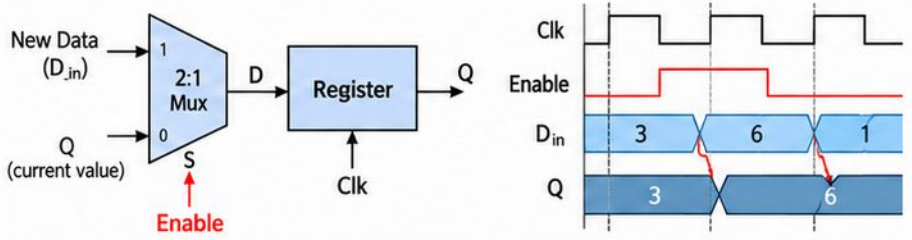
Write = 1 → update at clock edge; Write = 0 → keep value.

Register with Reset (3/5)



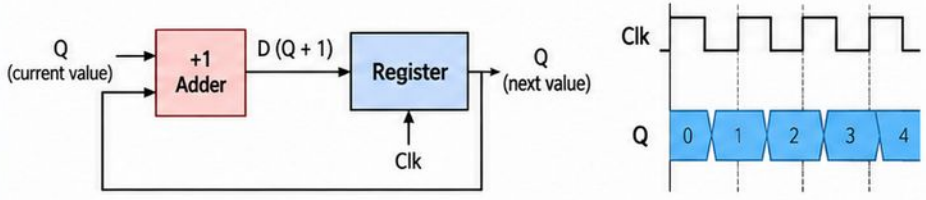
Reset = 1 → load predefined value (e.g., 0).

Register with Enable (Hold) (4/5)



Enable = 0 → hold current value; Enable = 1 → update at clock edge.

Register with Combinational Logic: Counter (5/5)



A register combined with an adder can form a counter that increments every clock.

Sequential Elements

Q4. A register initially stores Q = 4. At the next three rising clock edges, the following values are observed:

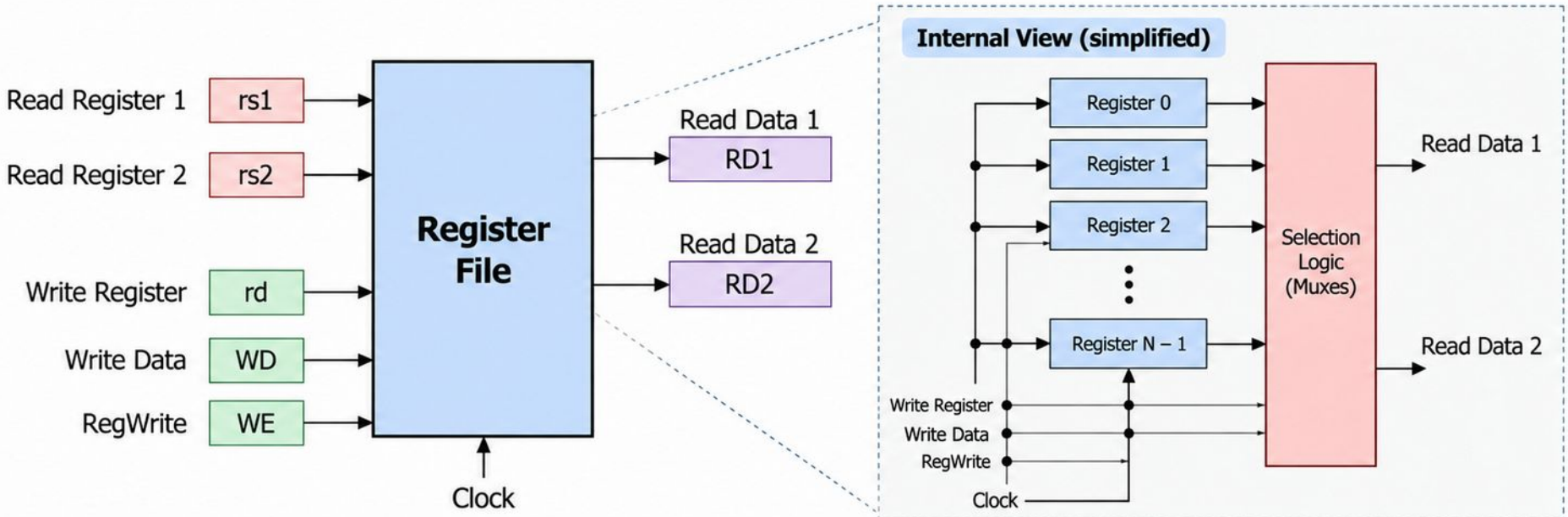
Rising Edge	D	Write
1	7	1
2	9	0
3	3	1

What is the value of Q after each rising edge?

A4.

Rising Edge	D	Write	Q after the edge
Initial	—	—	4
1	7	1	7
2	9	0	7
3	3	1	3

Register File



Read Operation

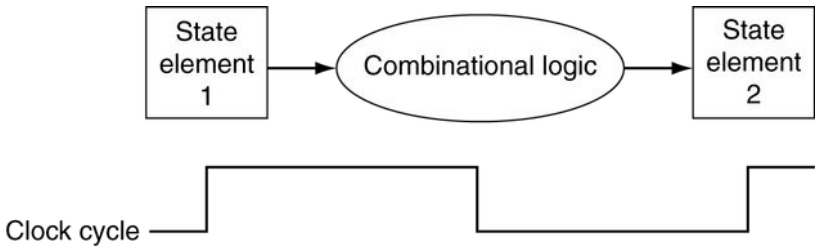
1. Specify a register using Read Register 1 or 2.
2. The register's stored value is provided as Read Data 1 or 2.
3. Reading does not change the stored value.

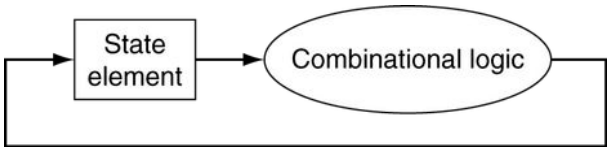
Write Operation

1. Specify a register using Write Register.
2. Provide the new value using Write Data.
3. When RegWrite = 1, the selected register is updated at the clock edge.

Clocking Methodology

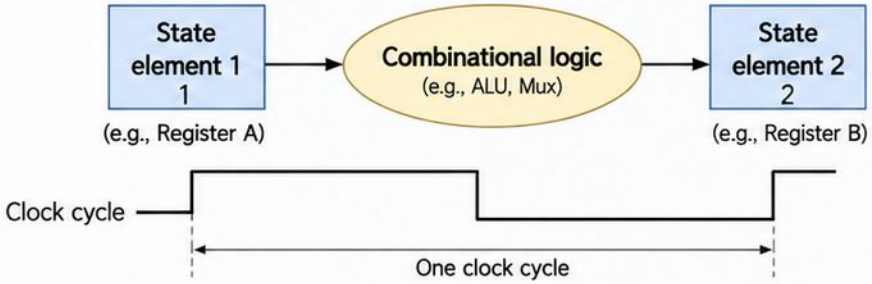
- Combinational logic transforms data during clock cycles
  - Between clock edges
  - Input from state elements, output to state element
  - Longest delay determines clock period





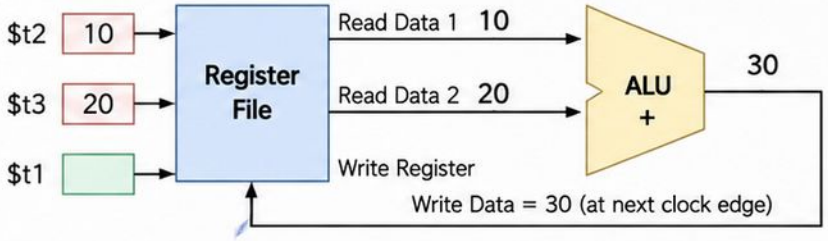
Clocking Methodology (Cont.)

1. State Element → Combinational Logic → State Element
(Transfer data between state elements)



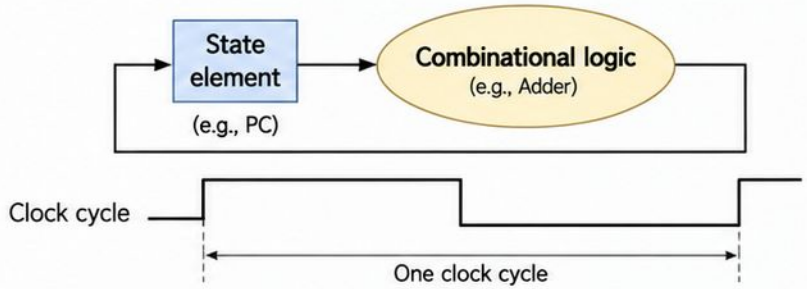
- The value stored in State element 1 is used as input to the combinational logic.
- The result is stored in State element 2 at the next clock edge.
- Used to transfer and process data between different registers or state elements.

Example: R-type Instruction (add $t1, $t2, $t3)



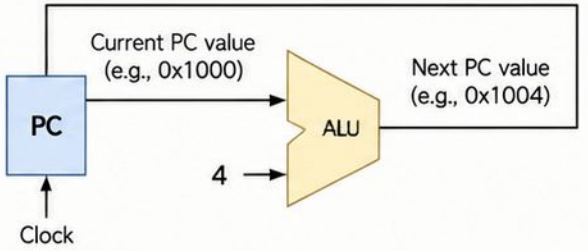
The value in $t2 and $t3 are read, added by the ALU, and written to $t1 at the next clock edge.

2. State Element → Combinational Logic → Same State Element
(Feedback to update the same state element)



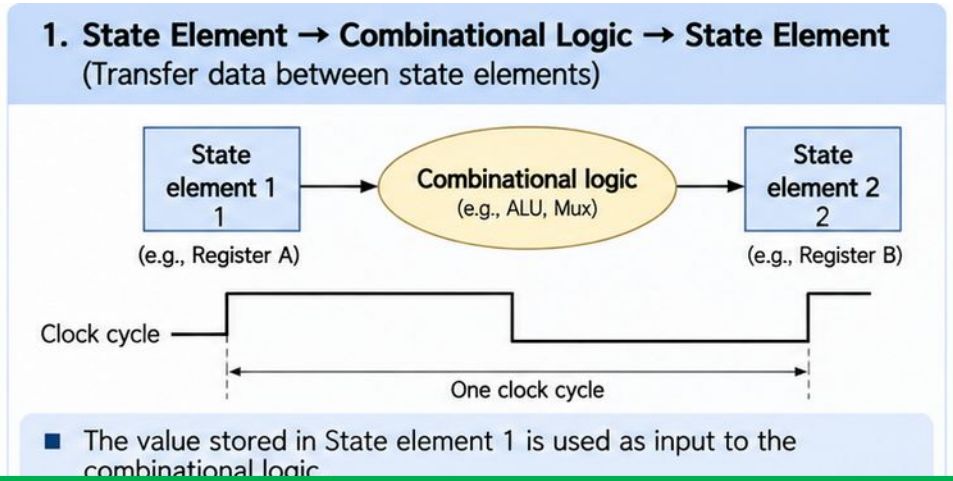
- The current value stored in the state element is used as input to the combinational logic.
- The result is written back to the same state element at the next clock edge.
- Used to update a value repeatedly (feedback).

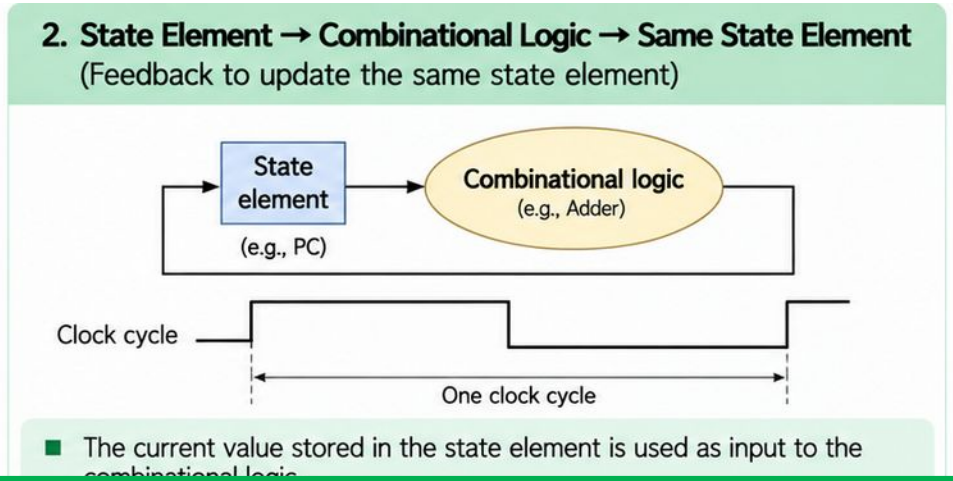
Example: Instruction Fetch (PC update)



After each clock edge, the PC is updated with PC + 4, so that the next instruction can be fetched.

Clocking Methodology (Cont.)





Q5. Consider the following processor operations:

(a)An add instruction reads two register values, adds them using the ALU, and stores the result in another register.

(b) After fetching an instruction, the processor computes PC + 4 and stores the result back into the PC.

Which clocking structure is used in each case?

A5.

(a)State Element → Combinational Logic → Different State Element

(b)State Element → Combinational Logic → Same State Element

Section 3-2 Quick Questions

Q1. Which hardware is used to compute, store, and transfer information in a processor?

Q2. Which combinational elements are used for addition, selection, and comparison?

Q3. For a 1-bit Half Adder, what are the Boolean expressions for Sum and Carry?

Q4. How do clock edges and Write control determine when a register updates its value?

Q5. How do the two clocking structures differ: writing to a different state element vs. writing back to the same state element?

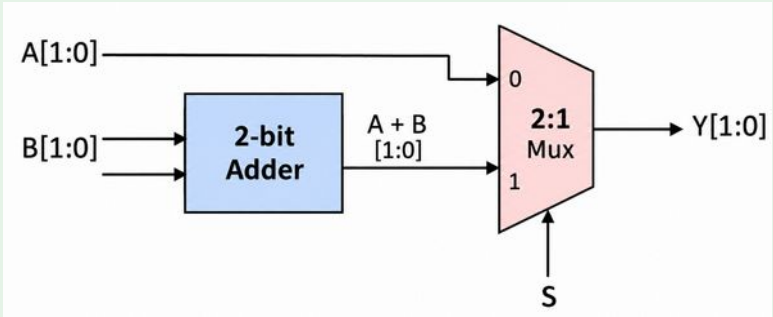
Exercise 1 — Combinational Circuit Design

A circuit has two 2-bit inputs, A and B.
Output A when S = 1.
(c) Find the output for A = 01, B = 10.
(a) Which combinational elements are required?
(b) Draw the circuit.
Output A + B when S = 0;

Answer

(a) Adder + Multiplexer (Mux)

(b) Circuit



(c) S = 0 → 01 + 10 = 11
S = 1 → Output = 01

Combine simple combinational elements to implement a required function.

Exercise 2 — Register File Operation

A register file initially contains the values below. Read R1 and R3, add them, and write the result into R4 with RegWrite = 1.

Register	R1	R2	R3	R4
Initial Value	5	8	12	20

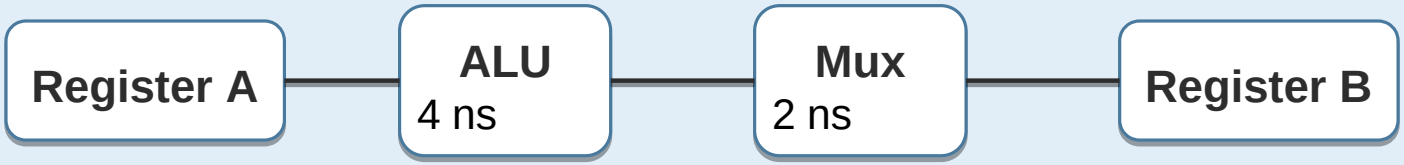
Answer

(a) Read: R1 = 5 and R3 = 12
(b) Adder output: 5 + 12 = 17
(c) RegWrite = 1 → R4 is updated to 17
Final: R1 = 5, R2 = 8, R3 = 12, R4 = 17

Register File operation: Read → Compute → Write.

Exercise 3 — Clock Period & Delay

Consider the clocked data path below. Ignore register-related delays.



(a) Total delay? (b) Minimum clock period? (c) What if Tclk = 5 ns?

Answer

(a) Total combinational delay = 4 ns + 2 ns = 6 ns

(b) Minimum clock period = 6 ns

(c) 5 ns < 6 ns → the result may not reach Register B before the ne xt clock edge.

The longest combinational delay determines the minimum clock period.